МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №1

по дисциплине

«Информационные и технические средства защиты авторских прав»

«Шифрование методами перестановок и подстановок»

Выполнил:
Лебедевич Артем Владимирович
магистрант группы 556301

Преподаватель:
Ярмолик Вячеслав Николаевич
доктор технических наук
профессор кафедры ПОИТ

Минск, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 1

2 Теоретические сведения 2
2.1 Общая модель перестановочного шифрования 3
2.2 Метод «железнодорожная изгородь» 3
2.3 Столбцовый метод 4
2.4 Метод поворачивающейся решётки 5
2.5 Шифр Цезаря и аффинное преобразование 6
2.6 Шифр Плейфейра 6
2.7 Шифр Виженера 7

3 Реализация 7
3.1 Структура программы 8
3.2 Ключевые фрагменты кода 8
3.3 Интерфейс командной строки 10
3.4 Графическое приложение 11

4 Результаты работы программы 11

5 Сравнение методов и стойкость 13

Заключение 15

Список использованных источников 16

1 ЦЕЛЬ РАБОТЫ

Изучить простейшие алгоритмы шифрования — методы перестановок и методы подстановок. Реализовать шифраторы и дешифраторы на основе трёх перестановочных методов («железнодорожная изгородь», столбцовый метод, поворачивающаяся решётка), а также подстановочных шифров по всем трём вариантам задания: шифр Цезаря, шифр Плейфейра и шифр Виженера. Проверить корректность реализации на примерах из методического пособия.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Общая модель перестановочного шифрования

При шифровании методами перестановок исходное сообщение M делится на блоки по d символов $M = m_1 \dots m_d,\ m_{d+1} \dots m_{2d},\ \dots$, после чего символы внутри блока переставляются в соответствии с некоторым вектором, функцией или фигурой:

$$E_k(M) = m_{f(1)} \dots m_{f(d)},\ m_{f(d+1)} \dots m_{f(2d)},\ \dots \tag{2.1}$$

Ключом служит сама перестановка f, дешифрование — применение обратной перестановки f^{-1}, поэтому шифр является симметричным. Принципиальное свойство перестановки: символы не заменяются, а лишь меняются местами, следовательно частоты букв шифротекста в точности совпадают с частотами букв исходного текста. Это делает перестановочные шифры уязвимыми к криптоанализу, но одновременно полезными в качестве слоя диффузии в составе современных блочных шифров.

Перед шифрованием текст нормализуется: отбрасываются пробелы и знаки препинания, буквы приводятся к верхнему регистру, буква «Ё» отождествляется с «Е». Шифротекст принято выводить группами по пять символов.

2.2 Метод «железнодорожная изгородь»

Символы исходного текста записываются в виде, напоминающем по форме забор (зигзагом по K строкам), а символы шифротекста считываются из полученной записи построчно. Высота изгороди K является ключом. Номер строки для позиции i определяется периодом $P = 2K - 2$:

$$r(i) = \begin{cases} p, & p < K, \\ P - p, & p \geqslant K, \end{cases} \qquad p = i \bmod P. \tag{2.2}$$

Для $M =$ «Это лабораторная работа по КиОКИ» и $K = 4$ запись имеет вид, приведённый в таблице 2.1, а шифротекст получается чтением строк сверху вниз: $C =$ «ЭОНОИ ТБРРА БТКОО ААОЯА АОКЛТ РПИ».

Таблица 2.1 – Запись текста зигзагом при $K = 4$

Строка	Символы
0	Э О Н О И
1	Т Б Р Р А Б Т К О
2	О А А О Я А А О К
3	Л Т Р П И

Число различных ключей равно числу допустимых значений K, то есть шифр вскрывается полным перебором высоты изгороди за время, линейное по длине текста.

2.3 Столбцовый метод

Исходное сообщение записывается в таблицу построчно, а затем считывается оттуда столбцами согласно вектору, задающему порядок считывания. Вектор задаётся ключевым словом: буквам ключа назначаются номера в соответствии с алфавитом, а если буква встречается несколько раз, нумерация определяется порядком следования повторяющихся букв в ключевом слове. Для ключа K = «КРИПТОГРАФИЯ» получаем вектор $(5, 8, 3, 7, 10, 6, 2, 9, 1, 11, 4, 12)$, а таблица шифрования приведена в таблице 2.2.

Таблица 2.2 – Шифрование столбцовым методом с ключом «КРИПТОГРАФИЯ»

5	8	3	7	10	6	2	9	1	11	4	12
Э	Т	О	Л	А	Б	О	Р	А	Т	О	Р
Н	А	Я	Р	А	Б	О	Т	А	П	О	К
И	О	К	И								

Считывая столбцы в порядке возрастания номеров, получаем C = «ААООО ЯКООЭ НИББЛ РИТАО РТААТ ПРК». Число ключей равно $d!$, где d — длина ключевого слова, однако шифр вскрывается методом анаграммирования: столбцы сопоставляются между собой по частотам биграмм русского языка.

Во время Первой мировой войны немецкие военные использовали *двойной столбцовый шифр*, при котором одно сообщение шифровалось дважды с одним и тем же или с двумя разными ключами. Для второго ключа K_2 = «ШИФРОВАНИЕ» с вектором $(10, 4, 9, 8, 7, 2, 1, 6, 5, 3)$ получается C = «КИРЯР ПЭОАИ ТОАОТ КОЛТО БАОБА АНР». Двойная перестановка разрушает столбцы первой таблицы и существенно усложняет анаграм-

мирование, однако и она была вскрыта: французской разведке требовалось несколько дней после смены ключа.

2.4 Метод поворачивающейся решётки

Исходный текст записывается на бумаге через отверстия решётки, которая по мере заполнения поворачивается на 90°. Решётка строится так: в матрице $n \times n$ (n — чётное) ячейкам, занимающим при повороте на 90° одинаковое положение, присваивается один и тот же номер, после чего вырезается по одному квадрату для каждого номера. При повороте клетка (r, c) переходит в клетку

$$(r, c) \longrightarrow (c,\ n - 1 - r), \tag{2.3}$$

то есть все клетки матрицы разбиваются на $n^2/4$ орбит по четыре клетки. Выбор ровно одной клетки из каждой орбиты гарантирует, что за четыре поворота решётка покроет каждую клетку матрицы ровно один раз.

Разметка матрицы 4×4 и использованная решётка приведены в таблице 2.3. Пустые клетки в конце заполняются ничего не значащими буквами.

Таблица 2.3 – Разметка орбит поворота и отверстия решётки 4×4

Разметка				Отверстия			
1	2	3	1	■	□	□	□
3	4	4	2	□	□	□	■
2	4	4	3	□	□	■	□
1	3	2	1	□	■	□	□

Для $M =$ «ШИФРОТЕКСТ» порядок заполнения матрицы по четырём поворотам приведён в таблице 2.4; жирным выделены буквы-заполнители.

Таблица 2.4 – Заполнение матрицы по поворотам решётки

Поворот	Клетки (r, c)	Записывается
0°	(0,0) (1,3) (2,2) (3,1)	Ш И Ф Р
90°	(0,3) (1,0) (2,1) (3,2)	О Т Е К
180°	(0,2) (1,1) (2,0) (3,3)	С Т **А Б**
270°	(0,1) (1,2) (2,3) (3,0)	**В Г Д Е**

Итоговая матрица считывается построчно, что даёт $C =$ «ШВСОТ ТГИАЕ ФДЕРК Б». Число ключей для решётки $n \times n$ составляет $4^{n^2/4}$: для решётки 4×4 это всего 256 вариантов, а для решётки 8×8 — уже $4^{16} \approx 4{,}3 \cdot 10^9$.

2.5 Шифр Цезаря и аффинное преобразование

Подстановочные шифры, в отличие от перестановочных, заменяют символы, сохраняя их позиции. Шифр Цезаря заменяет каждый символ алфавита исходного текста на сдвинутый относительно него на k позиций символ того же алфавита:

$$E_k(i) = (i + k) \bmod n, \qquad D_k(i) = (i + n - k) \bmod n. \tag{2.4}$$

Обобщением является метод децимации $E_k(i) = (i \cdot k) \bmod n$ и объединяющее оба метода аффинное преобразование с двумя ключами:

$$E_k(i) = (i \cdot k_1 + k_2) \bmod n, \qquad D_k(j) = k_1^{-1}(j - k_2) \bmod n. \tag{2.5}$$

Числа k_1 и n должны быть взаимно простыми: в противном случае обратного элемента k_1^{-1} не существует и разные буквы исходного текста склеиваются в одну, то есть преобразование перестаёт быть обратимым. В работе реализовано именно преобразование (2.5); классический шифр Цезаря получается при $k_1 = 1$. Мощность алфавита составляет $n = 26$ для английского языка и $n = 32$ для русского (буква «Ё» отождествлена с «Е»).

2.6 Шифр Плейфейра

Биграммный шифр Плейфейра применялся в Великобритании во время Первой мировой войны и является наиболее известным полиграммным шифром замены. Основой шифра служит шифрующая таблица, которая представляет собой сеансовый ключ; для удобства запоминания она заполняется ключевым словом, а затем оставшимися буквами алфавита. Для английского языка используется таблица 5×5, в которой позиция для символа «J» соответствует позиции символа «I» (таблица 2.5).

Таблица 2.5 – Шифрующая таблица для ключа «CRYPTOGRAPHY»

C	R	Y	P	T
O	G	A	H	B
D	E	F	I/J	K
L	M	N	Q	S
U	V	W	X	Z

Исходный текст разбивается на биграммы $m_i m_{i+1}$, которые преобразуются по правилам:

1. буквы в одной строке — берутся два символа справа от них (первый столбец считается столбцом справа по отношению к последнему): YT $\rightarrow$ PC;

2. буквы в одном столбце — берутся символы ниже (первая строка считается строкой ниже последней): VG $\to$ RE;
3. буквы в разных строках и столбцах — берутся два других угла прямоугольника, причём c_i находится в той же строке, что и m_i: ZF $\to$ WK;
4. при $m_i = m_{i+1}$ между ними вставляется пустой символ «X»: SS $\to$ SXS $\to$ QZ;
5. при нечётной длине текста пустой символ добавляется в конец.

Для M = «CIPHERTEXT» шифрование даёт C = «PDHIMGRKZP». Шифрование биграммами существенно повышает стойкость по сравнению с простой заменой, поскольку выравнивает частотные характеристики, однако частотные свойства распределения биграмм по-прежнему остаются ключом для злоумышленника.

2.7 Шифр Виженера

Шифр Виженера является многоалфавитным шифром подстановки и развивает идею Цезаря: таблицы подстановки чередуются в зависимости от последовательности символов ключа. Каждая строка таблицы Виженера представляет собой шифр Цезаря для конкретного значения ключа, поэтому шифрование описывается выражением

$$E_k(i) = (m_i + k_{i \bmod d}) \bmod n, \tag{2.6}$$

где d — длина ключа. Различают три варианта использования ключа:

1. **прямое использование** — ключ циклически повторяется;
2. **прогрессивный ключ** — при повторном применении символы ключа циклически сдвигаются на одну позицию в упорядоченном алфавите: «MODE», «NPEF», «OQFG» и так далее;
3. **самогенерирующийся ключ** — в качестве последующих символов ключа используется сам исходный текст.

Результаты шифрования слова M = «cryptography» ключом «MODE» по всем трём схемам приведены в таблице 2.6.

Таблица 2.6 – Шифр Виженера при различных схемах применения ключа

Схема	Последовательность ключа	Шифротекст
Прямая	MODEMODEMODE	OFBTF CJVMD KC
Прогрессивная	MODENPEFOQFG	OFBTG DKWOF ME
Самогенерирующаяся	MODECRYPTOGR	OFBTV FEGTD NP

3 РЕАЛИЗАЦИЯ

3.1 Структура программы

Программа написана на языке Python 3 без внешних зависимостей и разделена на модули по методам шифрования (листинг 3.1).

Листинг 3.1 – Структура проекта

```
main.py              CLI: выбор метода, режима и ключа
configs/cfg.py       ключи и параметры по умолчанию
helpers/text.py      нормализация текста и вывод группами по пять символов
helpers/keyword.py   числовой ключ (порядок столбцов) из ключевого слова
helpers/alphabet.py  выбор рабочего алфавита: русский (32) или английский (26)
lab1/fence.py        железнодорожная изгородь
lab1/columnar.py     столбцовый метод и двойная перестановка
lab1/grille.py       поворачивающаяся решётка
lab1/caesar.py       шифр Цезаря и аффинное преобразование
lab1/playfair.py     биграммный шифр Плейфейра
lab1/vigenere.py     шифр Виженера (три схемы ключа)
lab1/runner.py       прогон всех примеров методички с проверкой
gui/methods.py       описание шифров для окна: параметры, вызовы, разбор
gui/app.py           виджеты: выбор шифра, поля параметров, решётка
```

Во всех шифрах дешифратор не дублирует логику шифратора, а переиспользует расчёт позиций или сдвигов, выполняя обратное преобразование. Это гарантирует согласованность пары «шифрование–дешифрование» и вдвое сокращает объём кода.

3.2 Ключевые фрагменты кода

Расчёт номера строки зигзага (листинг 3.2) является общей основой шифрования и дешифрования для метода изгороди: шифратор читает символы по строкам, дешифратор раскладывает их обратно по тем же позициям.

Листинг 3.2 – Метод «железнодорожная изгородь»

```
def row_pattern(length: int, key: int) -> List[int]:
    """Номер строки для каждой позиции текста: 0,1,..,K-1,K-2,..,1,0,1,..."""
    if key < 2:
        return [0] * length
    period = 2 * key - 2
    return [p if p < key else period - p for p in (i % period for i in range(length))]


def encrypt(text: str, key: int) -> str:
    rows = row_pattern(len(text), key)
    return "".join(ch for row in range(key)
                   for ch, r in zip(text, rows) if r == row)
```

Числовой ключ столбцового метода получается сортировкой позиций букв ключевого слова по паре «буква, позиция»: вторичный ключ сортировки обеспечивает требуемую нумерацию повторяющихся букв слева направо (листинг 3.3).

Листинг 3.3 – Порядок чтения столбцов из ключевого слова

```
def read_order(keyword: str) -> List[int]:
    letters = normalize(keyword)
    return sorted(range(len(letters)), key=lambda i: (letters[i], i))
```

При дешифровании столбцового метода необходимо учитывать, что последняя строка таблицы может быть неполной: первые $|C| \bmod d$ столбцов длиннее остальных на один символ. Без этого учёта весь текст сдвигается и восстановление становится невозможным (листинг 3.4).

Листинг 3.4 – Восстановление длин столбцов при дешифровании

```
    full, tail = divmod(len(cipher), width)
    lengths = [full + (1 if c < tail else 0) for c in range(width)]
```

Для решётки ключ проверяется на корректность через разметку орбит: если два отверстия попали в одну орбиту, при повороте они совпадут и часть матрицы останется незаполненной (листинг 3.5).

Листинг 3.5 – Разметка орбит поворота и порядок заполнения

```
def rotate(cell: Cell, n: int) -> Cell:
    """Поворот клетки на 90 градусов по часовой стрелке."""
    row, col = cell
    return col, n - 1 - row


def fill_order(holes: List[Cell], n: int) -> List[int]:
    """Позиции матрицы в порядке заполнения: 4 поворота, внутри - построчно."""
    order: List[int] = []
    current = list(holes)
    for _ in range(4):
        order.extend(row * n + col for row, col in sorted(current))
        current = [rotate(cell, n) for cell in current]
    return order
```

Аффинное преобразование реализовано одной функцией: при дешифровании множитель обращается по модулю мощности алфавита встроенной функцией `pow(mul, -1, n)` (листинг 3.6).

Листинг 3.6 – Шифр Цезаря как частный случай аффинного преобразования

```
    if decode:
        inverse = pow(mul, -1, n)
        code = lambda i: (inverse * (i - add)) % n
    else:
        code = lambda i: (i * mul + add) % n
```

Реализация шифра Плейфейра не привязана к таблице 5×5: размеры вычисляются из мощности алфавита, поэтому для русского языка автоматически строится таблица 4×8. Три правила шифрования и дешифрования различаются только направлением сдвига, что задаётся параметром `step` $= \pm 1$ (листинг 3.7).

Листинг 3.7 – Правила шифра Плейфейра

```
        (r1, c1), (r2, c2) = position[pair[0]], position[pair[1]]
```

```
        if r1 == r2:                        # одна строка - сдвиг вбок
            c1, c2 = (c1 + step) % columns, (c2 + step) % columns
        elif c1 == c2:                      # один столбец - сдвиг вниз
            r1, r2 = (r1 + step) % rows, (r2 + step) % rows
        else:                               # прямоугольник - обмен столбцами
            c1, c2 = c2, c1
```

Три схемы применения ключа Виженера сведены к одной функции расчёта сдвига (листинг 3.8). Особенность самогенерирующегося ключа в том, что при дешифровании ключ восстанавливается по ходу работы: очередной его символ берётся из уже расшифрованной части текста, поэтому дешифратор передаёт в функцию накопленный результат, а шифратор — исходный текст.

Листинг 3.8 – Три схемы применения ключа Виженера

```
def _key_shift(i: int, key: Sequence[int], mode: str,
               plain: Sequence[str], alphabet: str) -> int:
    """Сдвиг для позиции i; для autokey используется уже известный текст."""
    length = len(key)
    if mode == "autokey":
        return key[i] if i < length else alphabet.index(plain[i - length])
    step = i // length if mode == "progressive" else 0
    return (key[i % length] + step) % len(alphabet)
```

3.3 Интерфейс командной строки

Программа управляется из командной строки: первый аргумент задаёт метод, второй — режим (`enc` — шифрование, `dec` — дешифрование), далее указываются ключ и текст. Текст может быть передан флагом `-t` или через стандартный ввод, флаг `--raw` отключает разбиение вывода на группы по пять символов (листинг 3.9).

Листинг 3.9 – Примеры запуска

```
python3 main.py demo

python3 main.py fence    enc -k 4 -t "Это лабораторная работа по КиОКИ"
python3 main.py column   enc -k КРИПТОГРАФИЯ -t "Это лабораторная работа по КиОКИ"
python3 main.py column   enc -k КРИПТОГРАФИЯ -d ШИФРОВАНИЕ -t "Это лабораторная..."
python3 main.py grille   enc -k 0,7,10,13 -t "ШИФРОТЕКСТ"

python3 main.py caesar   enc -k 3 -t "Это лабораторная работа по КиОКИ"
python3 main.py caesar   enc -k 5 -m 3 -t "Cryptography"
python3 main.py playfair enc -k CRYPTOGRAPHY -t "ciphertext"
python3 main.py vigenere enc -k MODE -s progressive -t "Cryptography"

echo "ШВСОТТГИАЕФДЕРКБ" | python3 main.py grille dec
```

Некорректные ключи отсекаются с понятным сообщением: перекрывающиеся при повороте отверстия решётки, множитель аффинного шифра, не взаимно простой с мощностью алфавита, символы ключа вне рабочего алфавита.

3.4 Графическое приложение

Помимо командной строки реализовано окно на библиотеке `tkinter` (входит в стандартную поставку Python, дополнительных зависимостей не требуется), запускаемое командой `python3 main.py gui`. Приложение построено так, чтобы параметры шифрования были открыты для изменения, а шифры выбирались по одному, а не применялись все сразу.

Окно состоит из четырёх частей:

1. выбор шифра из списка и общие настройки вывода (размер группы символов);
2. панель параметров, которая перестраивается под выбранный шифр;
3. два поля текста — открытый текст и шифротекст — с кнопками прямого и обратного преобразования;
4. панель «Как это работает», показывающая промежуточное представление: зигзаг изгороди, таблицу столбцов с номерами, разметку орбит и порядок заполнения решётки, шифрующую таблицу Плейфейра или последовательность ключа Виженера под текстом.

Чтобы интерфейс не приходилось править при добавлении нового шифра, описание шифров вынесено в отдельный модуль: каждый шифр задаётся записью `Method` со списком параметров `Param` и функциями шифрования, дешифрования и разбора (листинг 3.10). Виджет для параметра выбирается по его типу (`int` — поле с шагом, `text` — строка ввода, `choice` — выпадающий список, `grille` — интерактивная решётка), поэтому добавление шифра сводится к одной записи в словаре `METHODS`.

Листинг 3.10 – Описание шифра для интерфейса

```
    "Шифр Виженера": Method(
        title="Подстановка: каждая позиция шифруется своим сдвигом из ключа",
        params=[
            Param("keyword", "Ключевое слово", "text", "MODE"),
            Param("scheme", "Схема ключа", "choice", "direct", choices=vigenere.MODES),
        ],
        encrypt=lambda text, v: vigenere.encrypt(text, v["keyword"], v["scheme"]),
        decrypt=lambda text, v: vigenere.decrypt(text, v["keyword"], v["scheme"]),
        explain=_explain_vigenere,
    ),
```

Отдельно решена задача ввода ключа поворачивающейся решётки: вместо списка номеров клетки выбираются мышью, а клик по клетке автоматически снимает выбор с ранее выбранной клетки той же орбиты. Благодаря этому ключ остаётся корректным по построению — в нём всегда ровно одно отверстие на орбиту, и невозможно задать решётку, которая оставит часть матрицы незаполненной.

4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Команда `python3 main.py demo` выполняет все реализованные шифры на примерах из методического пособия и сравнивает полученные шифротексты с эталонными. Сводные результаты приведены в таблице 4.1, полный вывод программы — в листинге 4.1.

Таблица 4.1 – Результаты шифрования

Метод	Ключ	Совпадение
Железнодорожная изгородь	$K = 4$	да
Столбцовый метод	КРИПТОГРАФИЯ	да
Двойная перестановка	КРИПТОГРАФИЯ, ШИФРОВАНИЕ	да
Поворачивающаяся решётка	4×4, отверстия 0, 7, 10, 13	да
Шифр Цезаря	$k = 3$	эталона нет
Аффинное преобразование	$k_1 = 3$, $k_2 = 5$	эталона нет
Шифр Плейфейра	CRYPTOGRAPHY	да
Виженер, прямой ключ	MODE	да
Виженер, прогрессивный ключ	MODE	да
Виженер, самогенерирующийся ключ	MODE	да

Все восемь шифротекстов, для которых методическое пособие приводит эталонное значение, совпали с ним посимвольно. Дешифрование во всех случаях восстанавливает исходный текст с точностью до служебных символов — букв-заполнителей решётки и разделителей Плейфейра, которые получатель отбрасывает по смыслу сообщения.

Дополнительно выполнена проверка обратимости на случайных текстах: 300 текстов длиной до 60 символов для каждого перестановочного метода при всех допустимых значениях ключа и 400 текстов для подстановочных методов при всех сдвигах, всех допустимых множителях аффинного шифра и всех трёх схемах ключа Виженера. Расхождений не зафиксировано.

Листинг 4.1 – Вывод программы

```
=== ЛР1: шифрование методами перестановок и подстановок ===

=== Перестановочные методы ===

--- Железнодорожная изгородь (K = 4) ---
M = ЭТОЛА БОРАТ ОРНАЯ РАБОТ АПОКИ ОКИ
C = ЭОНОИ ТБРРА БТКОО ААОЯА АОКЛТ РПИ
D = ЭТОЛА БОРАТ ОРНАЯ РАБОТ АПОКИ ОКИ
дешифрование корректно; совпадает с методичкой

Ключ КРИПТОГРАФИЯ -> 5 8 3 7 10 6 2 9 1 11 4 12
--- Столбцовый метод (K = КРИПТОГРАФИЯ) ---
M = ЭТОЛА БОРАТ ОРНАЯ РАБОТ АПОКИ ОКИ
```

```
C = ААООО ЯКООЭ НИББЛ РИТАО РТААТ ПРК
D = ЭТОЛА БОРАТ ОРНАЯ РАБОТ АПОКИ ОКИ
дешифрование корректно; совпадает с методичкой

Ключ ШИФРОВАНИЕ -> 10 4 9 8 7 2 1 6 5 3
--- Двойная перестановка (K1 = КРИПТОГРАФИЯ, K2 = ШИФРОВАНИЕ) ---
M = ЭТОЛА БОРАТ ОРНАЯ РАБОТ АПОКИ ОКИ
C = КИРЯР ПЭОАИ ТОАОТ КОЛТО БАОБА АНР
D = ЭТОЛА БОРАТ ОРНАЯ РАБОТ АПОКИ ОКИ
дешифрование корректно; совпадает с методичкой

--- Поворачивающаяся решётка (4x4, отверстия 0,7,10,13) ---
M = ШИФРО ТЕКСТ
C = ШВСОТ ТГИАЕ ФДЕРК Б
D = ШИФРО ТЕКСТ АБВГД Е
дешифрование корректно; совпадает с методичкой

=== Подстановочные методы ===

--- Шифр Цезаря (вариант №1) (k = 3) ---
M = ЭТОЛА БОРАТ ОРНАЯ РАБОТ АПОКИ ОКИ
C = АХСОГ ДСУГХ СУРГВ УГДСХ ГТСНЛ СНЛ
D = ЭТОЛА БОРАТ ОРНАЯ РАБОТ АПОКИ ОКИ
дешифрование корректно

--- Аффинное преобразование (k1 = 3, k2 = 5) ---
M = CRYPT OGRAP HY
C = LEZYK VXEFY AZ
D = CRYPT OGRAP HY
дешифрование корректно

Шифрующая таблица для ключа CRYPTOGRAPHY:
C   R   Y   P   T
O   G   A   H   B
D   E   F   I/J K
L   M   N   Q   S
U   V   W   X   Z
--- Шифр Плейфейра (вариант №2) (K = CRYPTOGRAPHY) ---
M = CIPHE RTEXT
C = PDHIM GRKZP
D = CIPHE RTEXT
дешифрование корректно; совпадает с методичкой

--- Шифр Виженера, прямой ключ (K = MODE) ---
M = CRYPT OGRAP HY
C = OFBTF CJVMD KC
D = CRYPT OGRAP HY
дешифрование корректно; совпадает с методичкой

--- Шифр Виженера, прогрессивный ключ (K = MODE) ---
M = CRYPT OGRAP HY
C = OFBTG DKWOF ME
D = CRYPT OGRAP HY
дешифрование корректно; совпадает с методичкой

--- Шифр Виженера, самогенерирующийся ключ (K = MODE) ---
M = CRYPT OGRAP HY
C = OFBTV FEGTD NP
D = CRYPT OGRAP HY
дешифрование корректно; совпадает с методичкой
```

5 СРАВНЕНИЕ МЕТОДОВ И СТОЙКОСТЬ

Сводные характеристики реализованных шифров приведены в таблице 5.1, где N — длина текста, d — длина ключевого слова, n — мощность алфавита.

Таблица 5.1 – Сравнение реализованных шифров

Шифр	Ключей	Сложность	Способ вскрытия
Изгородь	N	$O(N)$	полный перебор высоты
Столбцовый	$d!$	$O(N)$	анаграммирование по биграммам
Двойной столбцовый	$(d!)^2$	$O(N)$	анаграммирование, требует больше текста
Решётка	$4^{n^2/4}$	$O(N)$	перебор при малом n, статистика текста
Цезарь	n	$O(N)$	полный перебор сдвига
Аффинный	$n \cdot \varphi(n)$	$O(N)$	частотный анализ, перебор
Плейфейр	$25!$	$O(N)$	частотный анализ биграмм
Виженер	n^d	$O(N)$	тест Казиски, индекс совпадений

Перестановочные шифры полностью сохраняют частоты одиночных букв, поэтому уже сам факт совпадения частотного профиля шифротекста с профилем языка указывает криптоаналитику на применённый класс шифра. Подстановочные шифры, наоборот, сохраняют позиции символов: одноалфавитная замена вскрывается частотным анализом, биграммный Плейфейр — анализом частот биграмм, а многоалфавитный Виженер сначала сводится к d независимым шифрам Цезаря определением периода ключа.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены простейшие алгоритмы шифрования и реализованы на языке Python шифраторы и дешифраторы трёх перестановочных методов («железнодорожная изгородь», столбцовый метод с одинарным и двойным ключом, поворачивающаяся решётка) и трёх подстановочных шифров, соответствующих всем вариантам задания (Цезарь с обобщением до аффинного преобразования, Плейфейр, Виженер с тремя схемами применения ключа). Корректность подтверждена совпадением с восемью эталонными примерами методического пособия и проверками обратимости на случайных данных.

Практический вывод работы состоит в том, что ни перестановка, ни подстановка по отдельности не обеспечивают криптостойкости: первая сохраняет частоты символов, вторая — их позиции, и обе вскрываются статистическими методами при доступном объёме шифротекста в несколько сотен символов. Значение этих преобразований в другом: в современных блочных шифрах (DES, AES) они применяются совместно и многораундово, где подстановка обеспечивает конфузию — нелинейную связь ключа и шифротекста, а перестановка обеспечивает диффузию — распространение влияния одного бита открытого текста на весь блок.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ярмолик, С. В. Криптография и охрана коммерческой информации: метод. пособие по выполнению лаб. работ / С. В. Ярмолик, В. Н. Ярмолик. — Минск: БГУИР, 2010. — 32 с.
2. Шнайер, Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си / Б. Шнайер. — М.: Триумф, 2002. — 816 с.
3. Столлингс, В. Криптография и защита сетей: принципы и практика / В. Столлингс. — М.: Вильямс, 2001. — 672 с.